
Detección de Fake News basados en Redes Convolucionales y Transformers

V. Aguilar, J.P. Fontana y L.A. Gasparutti

Facultad de Ingeniería y Ciencias Hídricas, Universidad Nacional del Litoral
Instituto de Investigación en Señales, Sistemas e Inteligencia Computacional, UNL-CONICET

Resumen

En éste trabajo hablaremos la detección automática de noticias falsas, una problemática que se fue intensificando con la evolución del internet, promoviendo la desinformación y desconexión de la realidad. Nuestro desafío como informáticos fue encontrar una manera de automatizar la detección de las noticias falsas, cuya escritura se va modificando con el paso del tiempo para inhibir este tipo de detecciones. Para abordar todo esto, trabajamos bajo dos propuestas: las redes convolucionales, variando los parámetros de las capas y modificando sus cantidades, y los transformers, utilizando un modelo pre entrenado adecuado para tareas de clasificación. Los resultados obtenidos de los distintos procesos nos dieron que las redes convolucionales alcanzan un 88.31 % de aciertos, confirmandose un buen funcionamiento del sistema. No obstante, los transformers obtuvieron tasas de aciertos del 95 %, lo que demuestra que es un método más fuerte para este tipo de problemas de clasificación.

1. Introducción

Hoy en día internet nos ha permitido acceder a la información mucho más rápido que antes y en cantidades nunca antes vistas, pero esa facilidad trajo también la intensificación de la problemática que trataremos en este trabajo: las noticias falsas (Fake News). Estas noticias promueven la desinformación, provocando consigo el conflicto de ideas y la desconexión con la realidad. Detectar cuales son falsas no es una tarea sencilla ya que, a medida que evolucionan los algoritmos para clasificarlas, la escritura de las fake news se modifica de tal forma que inhibe el trabajo de los programas que las detectan. Además, la utilización de modelos de lenguaje cada vez mas avanzados permite rapidamente transformar textos plagados de errores en verdaderas noticias creibles, aun cuando su informacion es completamente falsa. Como informáticos, nuestro desafío consiste en encontrar formas automáticas y/o mejorar algoritmos ya existentes para la clasificación de noticias, brindando una herramienta mas para combatir la divulgación de fake news. Este tipo de proyectos brindan una perspectiva fundamental, ya que si no se desarrollan los mismos, deberíamos revisar una por una cada noticia y clasificarlas según nuestra percepción, la cual podría estar sesgada o no muy definida.

De entre toda la bibliografía presente sobre esta problemática, tomamos como base un trabajo en particular que, según nuestro criterio, tenía resultados razonablemente buenos y reales (es decir, no tenía una tasa de aciertos cercana al 100 %), además de tener un dataset accesible y describir de manera precisa y detallada la metodología utilizada. El mismo se llama "Detection of Fake News by Convolutional Neural Networks and Recurrent Neural Networks" [1], realizado por Subhranil Das, Rashmi Kumari y Raghwendra Kishore Singh. En el trabajo mencionado se propuso resolver la detección de fake news usando una Red Neuronal Convolutiva junta a una Red Neuronal Recurrente. La arquitectura utilizada en dicho trabajo es: una CNN con 2 capas convolucionales y luego una RNN, más específicamente una, LSTM, luego todo se conecta a una red lineal con función sigmoidea que genera el resultado final. La tasa de aciertos más alta que obtuvieron con ésta arquitectura fue de 96.18 %. Nosotros tomamos de este trabajo el uso de las redes convolucionales

(en Resultados explicamos por qué descartamos las RNN) y, además, utilizamos transformers. Estos últimos no salieron de alguna bibliografía, sino de recomendaciones de nuestros docentes de la cátedra Inteligencia Computacional, los cuales nos lo recomendaron ya que son muy usados en trabajos de inteligencia cuando se trabaja con textos.

2. Métodos

Para comenzar con todo debemos trabajar la base de datos, lo que se conoce como “Preprocesamiento de datos”, ya que la misma cuenta con noticias vacías, caracteres corrompidos que solo entorpecen al reconocimiento, tags de HTML que no aportan información útil, etc. Algunas de las correcciones que hicimos son:

- Borrado de palabras entre corchetes [];
- Borrado de URLs (<https://...>);
- Borrado de comillas y espacios múltiples;
- Borrado de finales con puntos suspensivos;

Lo siguiente fue construir la matriz de embedding, la cual contiene como filas a los vectores numéricos que representan cada token (en este caso, a cada palabra, ya que nuestros tokens son directamente palabras). Para esto primero generamos una “matriz diccionario”, utilizando la que provee GloVe [2]. De este diccionario sacaremos los vectores de embedding para nuestros tokens. Luego, con un tokenizador, generamos nuestros tokens (sacados de la base de datos) y obtenemos sus respectivos vectores de embedding de la matriz GloVe. Cuando el token no se encuentra en la matriz diccionario, generamos su vector de embedding de manera aleatoria. Todos los vectores de embedding poseen el mismo tamaño, y la primera posición (primera fila) de la matriz de embedding es un vector de ceros correspondiente al padding.

Respecto al tokenizador, hay dos que tenemos en cuenta, el de Keras [3] y el de los transformers [4] (que es propio, pero podemos usarlo para las CNN también). Lo que nos hace determinar que, si estamos trabajando con las CNN, podemos usar tanto el Tokenizer de Keras como el de los Transformers. Sin embargo, con los transformers solo podemos ocupar el de los mismos.

Como se mencionó anteriormente, el trabajo lo tratamos con dos alternativas distintas, una con redes convolucionales y otra con transformers. Para un mejor desarrollo de cada una, dividiremos esta sección en dos, una para cada alternativa.

Las medidas de desempeño que usamos se basan en las que muestra el paper que usamos como guía:

- Accuracy: Es la tasa de aciertos, es decir, la cantidad de noticias bien clasificadas sobre el total de noticias que se evaluaron;
- BCE: Mide la diferencia entre la probabilidad predicha por el modelo y el resultado esperado (1/0).
- Precisión: Indica en qué proporción las noticias verdaderas fueron clasificadas como verdaderas.
- Recall: Indica en qué proporción nuestro modelo clasifica una noticia como verdadera.
- F1-score: Mide el balance entre la precisión y el recall. Es decir, que cuando sea alto, tendremos una buena tasa de detección de noticias verdaderas, y pocas noticias falsas tomadas como verdaderas.

2.1. Redes Convoluciones

Las redes convolucionales funcionan de manera similar al campo visual del cerebro, lo que permite reconocer patrones para poder obtener estímulos. Esto lo hace a través de filtros, con los cuales se realiza la operación de la convolución, entre las entradas de la capa y los filtros, y obtiene las salidas de la capa convolucional. Estas salidas tienen lo que denominamos “conocimiento local”, que hace referencia a que tienen información de los patrones determinados por el tamaño del filtro (kernel size). Son estos patrones los que nos permitirán detectar cuando una noticia es falsa o no.

Capa	Función
nn.Conv1d	Es la capa que realiza la operación de convolución entre las entradas y los filtros para generar las salidas.
nn.ReLU	Transforma las salidas a un formato binario (1/0), donde los números negativos pasan a 0 y los positivos a 1.
nn.MaxPool1d	Pool (máximo) reduce el tamaño de las salidas obtenidas, tomando de a grupos de tamaño kernel_size y quedándose con el mayor de ellos.
nn.Dropout	Basado en los trabajos de Hinton [6], lo que hace es desactivar algunas neuronas, forzando al resto a compensar el trabajo. Con esto evitamos el sobreentrenamiento.
nn.Sigmoid	Funcion sigmoidea, utilizada para la salida final del sistema (es decir, se usa como clasificador simple)

Tabla 1: Capas utilizadas para construir los modelos testeados

Para las redes convolucionales usamos las herramientas que nos brinda la librería de Pytorch [5], utilizando una estructura basada en 5 tipos de capas basicas (Tabla 1), combinandolas de distintas maneras.

Los datos que se usarán para el sistema están separados en tres conjuntos: entrenamiento, validación y testeo. Con los datos de entrenamiento son con los que la red define sus parámetros, luego se evalúa el sistema con la validación, y el entrenamiento se corta cuando la validación arroje una tasa de aciertos que no varíe más que una cierta tolerancia, cuando llegue al máximo de épocas, o cuando se alcance un accuracy deseado. Luego se pasa al testeo, que utiliza los datos de test guardados para probar cómo funciona el modelo ante datos que nunca antes había visto.

2.1.1. Algoritmos

Comenzamos generando las entradas de cada uno de los grupos que necesitamos (entrenamiento, validación y testeo), para los cuales definimos qué porcentaje de los datos totales va a cada grupo. Posteriormente agregamos los datos a un DataLoader, donde se agruparán en batches con los cuales trabajaremos para el entrenamiento, validación y testeo.

Creamos la red convolucional con la que trabajaremos enviando como parámetros:

- Los DataLoaders;
- La matriz de embedding;
- La cantidad de filtros;
- El tamaño de los filtros (kernel_size).

Dentro de la red se definen las subcapas y la cantidad de las mismas que vamos a usar. Por default, utilizamos una convolucional y una ReLu, que son las mínimas necesarias para funcionar. La convolucional se define con la cantidad de canales de entrada, cantidad de canales de salida, el tamaño de los filtros y el margen que se dejará en los bordes (esto último es porque no siempre el tamaño del filtro es divisor de la cantidad de elementos de la entrada, por ende quedarían datos sin trabajar). La capa de Pooling recibe el tamaño del kernel que usamos y el margen que se dejará. La capa de Dropout recibe el porcentaje de neuronas que se desactivan para forzar al resto a restaurar la salida.

El resultado final de las subcapas se convierte a un único vector plano que pasa a una capa lineal, la cual usa una función sigmoidea para generar la salida correspondiente a las entradas enviadas (obtenemos un 1 o 0 que determina la clasificación de la noticia).

Una vez generada toda la red, entramos en un bucle que corta por uno de tres motivos:

- Se alcanzó un límite de épocas (determinado por nosotros);
- Cuando se alcanza un valor de accuracy esperado;
- Cuando la diferencia entre accurancys obtenidas es menor a una tolerancia determinada.

En nuestro modelo se utiliza en un optimizador que nos brinda Pythorch, el cual se encargará de hacer el forward y backward de nuestro modelo, optimizando por batches que definimos en el Data-

Loader de entrenamiento. El optimizador que usamos se llama Adam, el cual tiene como principales ventajas el poder trabajar de manera óptima con muchos datos y reducir el impacto del decrecimiento del gradiente. Una vez terminados los batches de entrenamiento, utilizamos los de validación para calcular la tasa de aciertos del modelo actual. Cuando tenemos ese resultado, se evalúa con la tasa esperada o la tasa anterior para ver si debe cortar el bucle por llegar a tasa esperada o por la tolerancia respectivamente. En caso de que no se corte por ninguna de las anteriores, repetimos el bucle a partir del entrenamiento hasta que se cumpla alguna de las tres condiciones mencionadas anteriormente.

Luego de que el entrenamiento termine, usamos los datos de testeo (también en batches) para evaluar nuestro modelo con datos que jamás haya visto. Una vez terminado el testeo, mostramos las medidas de desempeño nombradas en la primera parte de la sección.

2.2. Transformers

Los transformers son redes neuronales que funcionan bajo el mecanismo de la autoatención. Este mecanismo permite relacionar los tokens que recibe como entrada y hacer foco en aquellos que generan los resultados esperados (es decir, presta atención a los patrones presentes en noticias falsas). La “atención” que se evalúa es equivalente al “producto interno” entre los vectores de embedding de los tokens que intervienen en la entrada que se está analizando.

El trabajo que se realiza con los transformers consta de dos subprocesos. Encode (toma una entrada y la convierte en una salida con significado contextual) y Decode (cuyo objetivo es generar una secuencia de salida). El modelo pre entrenado que nosotros usamos es DistilBERT, que es un modelo basado en otro modelo de Encode llamado BERT. Utilizamos un modelo únicamente de encode porque lo que buscamos es hacer una clasificación, y no generar un nuevo texto como salida.

La relación entre BERT y DistilBERT es maestro-alumno, donde BERT “enseña” a DistilBERT como predecir palabras basados en contextos, haciendo que DistilBERT no necesite de tantos parámetros para ser usado. En este sentido, se podría decir que el alumno aprende para ser tan bueno como el maestro. No siempre obtiene los mismos resultados, pero siempre son muy similares, y esto lo hace con mayor rapidez, eficiencia y además siendo más liviano. DistilBERT es el modelo pre entrenado que usamos para generar el tokenizador para el transformer con el que trabajamos.

2.2.1. Algoritmos

Utilizamos la librería Hugging Face para el trabajo de los transformers, ya que es la más utilizada hoy en día para este tipo de arquitecturas. Comenzamos pasando los datos preprocesados (como se explicó al principio de la sección Desarrollo) al formato de una base de datos de Hugging Face, separando en el proceso un sector de los datos para el entrenamiento y otro para el testeo.

Luego, cargamos el modelo pre entrenado DistilBertEncased y el tokenizador del modelo mencionado. Todo esto se hace con el formato de la biblioteca mencionada anteriormente, ya que eso nos provee funciones que nos permitirán manejar de manera más sencilla la red. Luego procedemos con el entrenamiento de la red, usando la base de datos de entrenamiento con datos etiquetados, donde nosotros tuvimos que aclarar con qué etiquetas reconoce a una noticia falsa de una verdadera. Finalmente realizamos el testing (con los datos separados para ese proceso), obteniendo las medidas de desempeño que la librería nos brindaba y que coincidían con las que nosotros trabajamos (Accuracy, F1-Score y BCE).

3. Resultados

3.1. Redes Convolucionales

Comenzamos tomando del total de datos, un 90 % para entrenamiento y el otro 10 % para testeo. Luego, de ese 90 % de entrenamiento, un 80 % son efectivamente para entrenamiento, y el 20 % restante es para validación. Luego, con cada grupo de datos formados, se arman las entradas, juntando los vectores de embedding de los tokens que forman las noticias. Para el entrenamiento propusimos un valor de aciertos esperados del 90 % y una tolerancia de 0.001. Estos valores los usamos para evaluar el corte del entrenamiento. El número máximo de épocas propuesto es de 50, siguiendo el modelo del paper que usamos como guía [1]. El entrenamiento lo hacemos por batches y usamos el tokenizador de los transformers. Para la red convolucional usamos 2 capas convolucionales (es decir,

2 elementos de los primeros 4 mencionados en la tabla 1). La primera capa convolucional tiene 100 canales de entrada y 128 de salida, con 128 filtros de tamaño 5, permitiendo un margen de 1 para los bordes. La segunda capa convolucional tiene 128 canales de entrada y 256 canales de salida, con 128 filtros de tamaño 5, permitiendo un margen de 1 para los bordes. Los resultados obtenidos con esta combinación son:

- Entrenamiento:
 - Accuracy: 0.857
 - BCE: 0.2693
 - Total de épocas: 8
- Testeo:
 - Accuracy: 0.8831
 - BCE: 0.3061
 - Precisión: 0.8605
 - Recall: 0.8892
 - F1-score: 0.8794

Estos resultados demuestran que el clasificador funciona bastante bien, ya que de cada 100 noticias, 88 serán clasificadas correctamente. Como se mencionó anteriormente, el hecho de que F1-score de un resultado cercano a uno, significa que nuestro sistema sabe clasificar bien cuales son las noticias reales. En la imagen siguiente se presenta una gráfica de la evolución del Accuracy obtenido durante el entrenamiento.

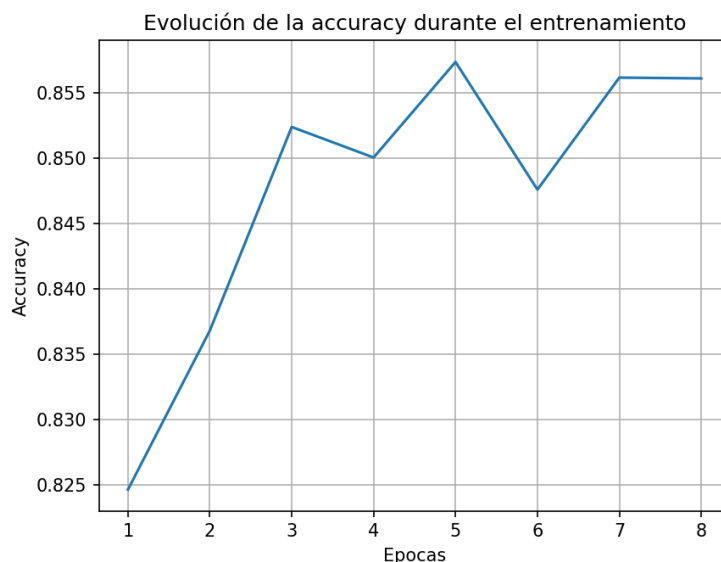


Figura 1: Evolución del accuracy durante el entrenamiento

En la Figura 1 vemos cómo varía el accuracy a medida que pasan las épocas. Podemos notar como no es un incremento monótono, sino que puede subir o bajar en el paso de las épocas. El pico más alto representa la mejor tasa de aciertos que obtuvimos a lo largo del entrenamiento. Si bien se presentan caídas en el accuracy, podemos destacar que hay una gran tendencia al incremento del mismo en las primeras 3 épocas, dando a entender que los mejores ajustes los tuvo en esas primeras épocas. Otra cosa que se puede notar es que, si agregamos más épocas, todo apuntaría a que el accuracy convergerá a valores cercanos a 0.855. Para comprobar si las clasificaciones eran mejores con más o menos capas convolucionales, realizamos pruebas con los mismos parámetros, pero variando la cantidad de capas. Los resultados se pueden observar en la Tabla 2.

En la Tabla 2 podemos observar las comparaciones entre accuracy y BCE (de entrenamiento y testeo) y el tiempo de ejecución de la red evaluada con distinta cantidad de capas convolucionales. Podemos

Capas	Accuracy (trn)	Accuracy (tst)	BCE (trn)	BCE (tst)	Tiempo de ejecución
1 capa	0.8459	0.8602	0.3262	0.3286	02:39.82
2 capas	0.8573	0.8709	0.2650	0.3033	03:30.60
3 capas	0.8524	0.8665	0.3251	0.3197	03:52.08
4 capas	0.8490	0.8649	0.3161	0.3265	06:04.05
5 capas	0.8459	0.8439	0.4699	0.3681	06:43.52

Tabla 2: Comparación de resultados para distintas cantidades de capas

observar que con dos capas, obtenemos el mayor accuracy y menor BCE, aunque su ejecución dura 1 minuto más que usando una sola capa. Esto nos permite ver que no es necesario aumentar la cantidad de capas para obtener los mejores resultados, todo lo contrario, estaríamos demorando más el proceso para obtener incluso resultados un poco peores.

Otros resultados a destacar:

- Cambiar la función de Pooling, entre máximo (elige el máximo de los datos dentro del filtro) y el promedio (hace el promedio de los datos dentro del filtro) es indiferente, ya que después de varias pruebas seguimos obteniendo resultados similares;
- Quitar la capa de DropOut elevaba bastante la tasa de aciertos del entrenamiento, pero baja la de testing. Esto se debe a que la capa de dropout es utilizada para evitar el sobreentrenamiento;
- Aumentar la cantidad de épocas máximas o reducir la tolerancia solo generaba que el algoritmo demore más en converger, ya que el resultado terminaba siendo el mismo que obtuvimos en la solución presentada al principio de la sección;
- Utilizamos el tokenizador de los transformers para trabajar y obtener resultados ya que, cuando comparamos los resultados obtenidos con el tokenizador de Keras, veíamos que estos últimos tenían una tasa de aciertos más baja:

- Tokenizador de transformers: el promedio de aciertos es 0.86;
- Tokenizador de Keras: el promedio de aciertos es 0.82.

Si bien los resultados no difieren demasiado, utilizamos el tokenizador de los transformers ya que, ante cualquier variación que le hacíamos la red, obtenía un mejor resultado que usando el otro tokenizador;

- Como se mencionó en la introducción, nosotros descartamos el uso de una red recurrente. Esto fue debido a que cuando la agregábamos, el accuracy de nuestro modelo descendía a valores cercanos a 0.30. La red que intentamos usar se llama GRU.

3.2. Transformers

Para el entrenamiento del transformer usamos un tasa de aprendizaje de 2×10^{-5} , batches de 16 elementos cada uno y 2 épocas de entrenamiento. La división de los datos de entrenamiento y testeo lo hace internamente el modelo. Utilizando estos datos, los resultados obtenidos fueron:

- Accuracy: 0.95;
- F1-score: 0.95;
- BCE: 0.136.

Estos resultados son muy buenos, ya que nos dice que nuestro modelo clasifica bien 95 de cada 100 noticias, lo cual supera los resultados obtenidos por las redes convolucionales. Más específicamente, F1-score nos está diciendo que nuestro modelo es muy bueno detectando correctamente una noticia verdadera. La Figura 2 representa a la matriz de atención de nuestro modelo, usando como frase de ejemplo “Trump STUPIDLY Attacks A Major U.S. Ally Before Threatening North Korea With War” (Noticia falsa generada en base a las palabras más repetidas en nuestra base de datos).

Lo que vemos en la Figura 2 es una matriz de colores que representa la atención que le da el Transformer a las palabras de una frase, a partir de una palabra que toma como referencia. El eje vertical

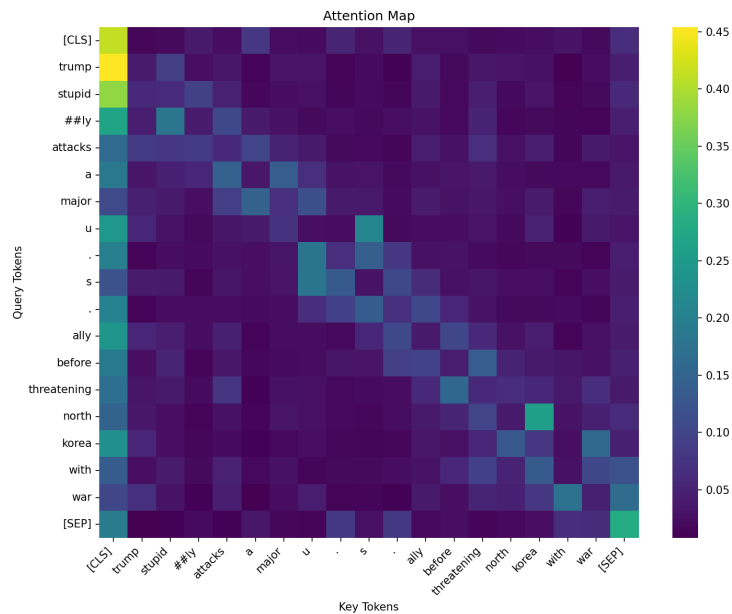


Figura 2: Mapa de atención del transformer para una entrada particular

representa los Query Tokens, que son los tokens que se toman como referencia para analizar la atención que se le da al resto respecto a él. En el eje horizontal están los Key Tokens que son, por cada Query Token, los valores de atención que le presta el transformer para determinar las salidas. Lo que hay que destacar del gráfico es la detección de patrones a los cuales el sistema le presta atención para detectar cuando una noticia es real o no. Por ejemplo: cuando tomamos como referencia el comienzo de la oración, presta mayor atención a la palabra “Trump”, o cuando tomamos la palabra “north”, presta mucha atención a la palabra “korea”.

4. Conclusiones

En conclusión, logramos implementar dos enfoques distintos de inteligencia artificial para la detección de las noticias falsas: las CNN y los Transformers. Respecto a las redes convolucionales, notamos que obtienen buenos resultados, ya que cuando hacíamos variar las capas entre 1 y 5, la tasa de aciertos variaba entre 0.84 y 0.88, y se obtienen valores altos del F1-score, lo cual significaba que nuestro clasificador tenía un alto balance entre la detección de noticias verdaderas y que verdaderamente esas noticias sean verdaderas. Además, se confirmó lo que Hinton decía en sus trabajos sobre la subcapa de Dropout, ya que tener esa subcapa garantiza que el modelo no se sobreentrena, obteniendo buenos resultados de testing, es decir, que funciona bien con datos que nunca vió. Respecto a los transformers, podemos concluir que son mejor modelo que las CNN para el trabajo de textos, ya que obtuvimos valores de tasas de aciertos del 95 % y valores de F1-score de 0.95, es decir, un gran balance entre la detección de noticias verdaderas y que realmente sean verdaderas. Además se puede concluir con las matrices de atención, que para la detección de noticias verdaderas no necesita generar patrones de palabras relacionadas muy largas, sino que hay un par de patrones específicos con los que ya puede tomar conclusiones sobre la veracidad de la noticia.

Agradecimientos

Agradecemos a L. Di Persia por las ideas y revisiones que mejoraron mucho esta guía. Este es un documento abierto y libre. Está abierto a incorporar más sugerencias o correcciones que nos envíen (que serán muy bienvenidas), y también está liberado a que se haga una copia, se modifique con cambios más de fondo, propios de otras disciplinas o formas de escribir, y se distribuya sin restricciones.

Referencias

1. Subhranil Das, Rashmi Kumari, Raghendra Kishore Singh, "Detection of Fake News by Convolutional Neural Networks and Recurrent Neural Networks",
<https://www.sciencedirect.com/science/article/pii/S1877050925015844>.
2. GloVe,
<https://github.com/stanfordnlp/GloVe>.
3. Keras: Tokenizer,
https://keras.io/keras_hub/api/tokenizers/tokenizer/.
4. Tokenizer de DistilBERT,
https://huggingface.co/docs/transformers/model_doc/distilbert.
5. CNN de Pytorch,
<https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv1d.html>.
6. Dropout y dichos de Hinton,
<https://educasmart.com/inteligencia-artificial/dropout/#:~:text=Hinton%20creador%20de%20la%20t%C3%A9cnica%20elige%20una%20arquitectura%20aleatoria.%C2%BB>.